CSCI 3271 — Operating Systems
Programming Project 4
Due: Friday October 30

Instructor: Edward Tremel
School of Computer and Cyber Sciences
Augusta University

Common Instructions: For this assignment you will need to write a program to solve a problem, based on handout code provided on D2L. You should upload your solution to D2L in individual plaintext files with the correct extension for source code (i.e. `.py` for Python, `.c` or `.h` for C). In general, you do not need to submit handout code files that are unchanged; you only need to upload code files that you modified in some way.

Project Description: This project focuses on the implementation of a file system called VSFS, or Very Simple File System, that is similar in design to the Linux ext2 file system but much simpler. Like ext2, it uses inodes to store metadata about where a file's data blocks are located, stores inodes in a table at the beginning of the disk partition, and keeps track of free inodes and free data blocks with bitmaps. Directory entries associate a name with an inode number, although the names have a fixed size (instead of using flexible-size directory entries). More information about the file system's design can be found in Chapter 40 of *Operating Systems: Three Easy Pieces* and in the README file included in the project folder on D2L.

The file system implementation is mostly complete already, and it can be run with a simulated block device that stores blocks either in memory or in a file in the host operating system's filesystem. Your task is to implement the **link** and **unlink** operations, filling in the `vsfs_link` and `vsfs_unlink` functions. These operations have the same semantics as they do in the ext2 file system: **link** creates a new directory entry for an existing file, while **unlink** removes a directory entry and potentially deletes the file it refers to (if that file's link count reaches 0).

Part 1: Describe the sequence of low-level operations that **link** and **unlink** need to perform, in terms of the blocks on the disk and the data structures of the file system. This will help you plan your implementation. Submit your response in a PDF or plain-text file, like other written responses.

Part 2: Write the `vsfs_link` and `vsfs_unlink` functions, and add code to the `test_main.c` file to test them to ensure they work.